

Module 1 : Java Moderne (Java 17 / 21)

Ce premier module est découpé en 3 chapitres clés :

1. **Chapitre 1 : Les nouveautés de Java** (Évolution, LTS, JDK/JRE/JVM, Bytecode & Garbage Collector)
2. **Chapitre 2 : Les Enums** (Concepts, syntaxe, attributs métier, méthodes intégrées & TP complet)
3. **Chapitre 3 : Les Records** (Syntaxe, suppression du code verbeux, validations compactes, limites & TP complet)

Chapitre 1 : Les nouveautés de Java

1.1 L'évolution du cycle de release de Java

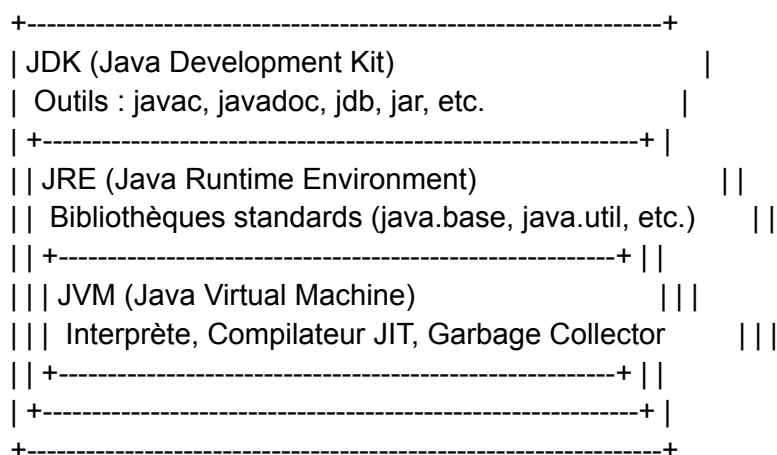
Pendant très longtemps, Oracle publiait une nouvelle version majeure de Java tous les 3 à 5 ans (ex: Java 6, Java 7, Java 8). Depuis Java 9, l'écosystème a adopté une stratégie de **release semestrielle** (une nouvelle version sort tous les 6 mois, en mars et en septembre).

Dans ce nouveau modèle, on distingue deux types de versions :

- **Versions Feature (Standard)** : Sortent tous les 6 mois et ne sont supportées que pendant 6 mois. Elles permettent d'expérimenter rapidement de nouvelles fonctionnalités.
- **Versions LTS (Long-Term Support)** : Sortent environ tous me 2 ans (ex: Java 8, 11, 17, 21). Elles bénéficient d'un support et de correctifs de sécurité entreprise pendant plusieurs années. **Java 17** et **Java 21** sont les deux standards actuels en entreprise.

1.2 Architecture de l'environnement Java : JDK vs JRE vs JVM

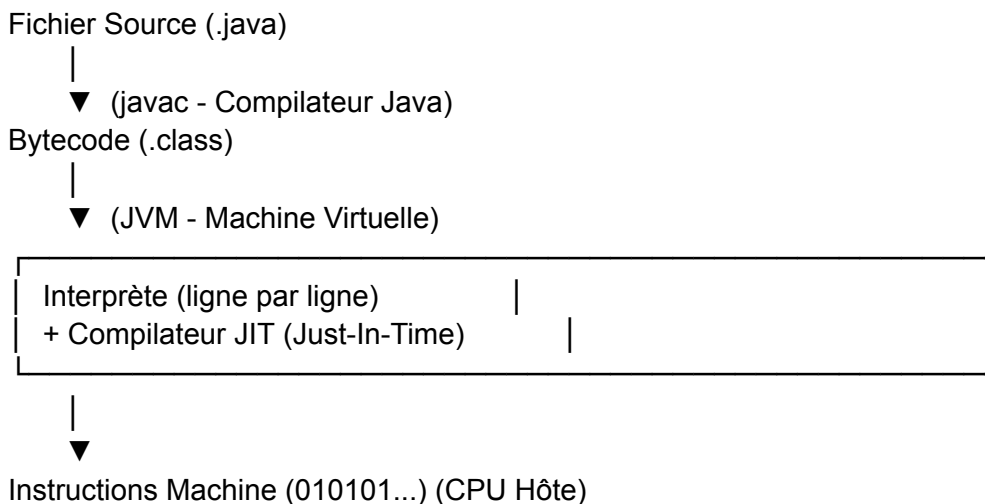
Pour comprendre comment tourne un programme Java, il faut maîtriser la différence entre ces trois composants :



1. **JDK (Java Development Kit)** : C'est la boîte à outils complète du développeur. Elle inclut le compilateur (**javac**), le compresseur de projet (**jar**), la documentation (**javadoc**), et le JRE.
2. **JRE (Java Runtime Environment)** : C'est l'environnement d'exécution nécessaire pour **lancer** une application Java déjà compilée. Il contient les bibliothèques de classes standards de Java.
3. **JVM (Java Virtual Machine)** : C'est le cœur d'exécution. La JVM est propre à chaque système d'exploitation (Windows, Linux, macOS). C'est elle qui charge, vérifie et exécute le *bytecode*.

1.3 Compilation, Bytecode et Compilateur JIT

Java est un langage hybride : il est à la fois **compilé** et **interprété**.



1. **Compilation initiale** : Le développeur écrit un fichier **.java**. La commande **javac MonProgramme.java** transforme le code en un fichier binaire intermédiaire appelé **Bytecode (MonProgramme.class)**. Ce bytecode est indépendant de la machine physique.
2. **Exécution** : La JVM lit ce bytecode.
3. **Compilateur JIT (Just-In-Time)** : Durant l'exécution, la JVM repère les sections de code très souvent exécutées (appelées *hotspots*). Le compilateur JIT traduit à la volée ce bytecode directement en instructions machine ultra-optimisées. Cela permet à Java d'atteindre des performances proches de C++.

1.4 Le Garbage Collector (GC)

En Java, le développeur n'a pas à gérer manuellement la mémoire (contrairement au C/C++ avec **malloc** ou **free**). La mémoire dynamique est gérée dans une zone appelée le **Heap** (Tas).

- **Son rôle** : Le Garbage Collector est un fil d'exécution (*thread*) en arrière-plan qui parcourt régulièrement le Heap pour identifier les objets qui ne possèdent plus aucune référence active et libérer la mémoire qu'ils occupaient.
- **Les algorithmes modernes** : Avec Java 17 et 21, des GCs de haute performance comme **G1GC** (par défaut) ou **ZGC** (Zero-Garbage Collection) permettent de traiter des gigaoctets de mémoire avec des pauses d'interruption quasi imperceptibles (inférieures à 1 ms).

Chapitre 2 : Les Enums (**enum**)

2.1 Pourquoi utiliser un Enum ?

Avant l'introduction des **enum**, on utilisait souvent des constantes de type chaîne de caractères (*magic strings*) ou entiers (*magic numbers*) :

```
Java
// ❌ Mauvaise pratique (Ancien Java)
public static final String STATUS_ACTIVE = "ACTIVE";
public static final String STATUS_BLOCKED = "BLOCKED";
```

Cette approche pose deux problèmes majeures :

1. **Aucune sécurité au typage** : Vous pouvez accidentellement passer n'importe quelle chaîne (ex: "**ACTIVEE**") à une méthode sans que le compilateur ne bronche.
2. **Pas de lisibilité ni d'auto-complétion.**

Un **Enum** définit un type de données fini et fermé.

2.2 Syntaxe de base et méthodes intégrées

```
Java
public enum Status {
    ACTIVE,
    BLOCKED,
    DELETED
}
```

Chaque constante d'un enum bénéficie automatiquement de méthodes fournies par la classe parente **java.lang.Enum** :

```
Java
public class TestEnum {
    public static void main(String[] args) {
        Status s = Status.ACTIVE;

        // 1. name() : Renvoie le nom exact de la constante sous forme de String
```

```

        System.out.println(s.name()); // "ACTIVE"

        // 2. ordinal() : Renvoie la position (index 0) dans la déclaration
        System.out.println(s.ordinal()); // 0

        // 3. values() : Renvoie un tableau contenant toutes les constantes
        for (Status st : Status.values()) {
            System.out.println(st);
        }

        // 4. valueOf(String name) : Convertit une chaîne en Enum (attention aux exceptions !)
        Status recherche = Status.valueOf("BLOCKED"); // Status.BLOCKED
    }
}

```

2.3 Enums avancés : Attributs, Constructeurs et Méthodes métier

En Java, un **enum** est en réalité une **classe spéciale**. Il peut donc posséder :

- Des attributs privés et immutables (**private final**).
- Un constructeur (toujours privé ou implicitement privé).
- Des méthodes métier.

Java

```

public enum Priority {
    LOW(1, "Basse priorité"),
    MEDIUM(2, "Priorité moyenne"),
    HIGH(3, "Haute priorité"),
    CRITICAL(4, "Priorité critique");

    private final int level;
    private final String description;

    // Constructeur d'enum (exécuté pour chaque constante)
    Priority(int level, String description) {
        this.level = level;
        this.description = description;
    }

    public int getLevel() {
        return level;
    }

    public String getDescription() {
        return description;
    }
}

```

```
// Méthode métier personnalisée
public boolean isUrgent() {
    return this == HIGH || this == CRITICAL;
}
}
```

TP 1 : Gestionnaire de Statuts de Commande (*OrderStatus*)

Objectif :

Créer un enum métier complet pour gérer les étapes de livraison d'une commande d'un site e-commerce et valider les transitions d'état.

Consignes du TP :

1. Créer un enum `OrderStatus` contenant les constantes :
 - `PENDING` (Code 100, Libellé "En attente de paiement")
 - `PAID` (Code 200, Libellé "Payée")
 - `SHIPPED` (Code 300, Libellé "Expédiée")
 - `DELIVERED` (Code 400, Libellé "Livrée")
 - `CANCELLED` (Code 500, Libellé "Annulée")
2. Déclarer les attributs `code` (int) et `label` (String).
3. Ajouter une méthode `boolean isTerminal()` qui retourne `true` si la commande est dans un état final (`DELIVERED` ou `CANCELLED`).
4. Ajouter une méthode `boolean canTransitionTo(OrderStatus nextStatus)` selon les règles suivantes :
 - Une commande dans un état terminal ne peut plus changer d'état.
 - `PENDING` ne peut passer qu'à `PAID` ou `CANCELLED`.
 - `PAID` ne peut passer qu'à `SHIPPED` ou `CANCELLED`.
 - `SHIPPED` ne peut passer qu'à `DELIVERED`.
5. Écrire une classe `Main` de test pour valider les comportements.

Correction détaillée du TP 1

Java

```
public enum OrderStatus {
    PENDING(100, "En attente de paiement"),
    PAID(200, "Payée"),
    SHIPPED(300, "Expédiée"),
    DELIVERED(400, "Livrée"),
    CANCELLED(500, "Annulée");

    private final int code;
    private final String label;

    OrderStatus(int code, String label) {
```

```

        this.code = code;
        this.label = label;
    }

    public int getCode() {
        return code;
    }

    public String getLabel() {
        return label;
    }

    // Indique si la commande est terminée
    public boolean isTerminal() {
        return this == DELIVERED || this == CANCELLED;
    }

    // Valide la transition logique vers le statut suivant
    public boolean canTransitionTo(OrderStatus nextStatus) {
        if (isTerminal()) {
            return false; // Impossible de modifier un état final
        }

        return switch (this) {
            case PENDING -> nextStatus == PAID || nextStatus == CANCELLED;
            case PAID -> nextStatus == SHIPPED || nextStatus == CANCELLED;
            case SHIPPED -> nextStatus == DELIVERED;
            default -> false;
        };
    }
}

```

Classe de test (Main.java) :

Java

```

public class MainTP1 {
    public static void main(String[] args) {
        OrderStatus current = OrderStatus.PENDING;

        System.out.println("Statut actuel : " + current.getLabel() + " (Code: " +
current.getCode() + ")");

        // Test de transition valide
        if (current.canTransitionTo(OrderStatus.PAID)) {
            current = OrderStatus.PAID;
            System.out.println("Transition réussie vers : " + current.getLabel());
        }
    }
}

```

```

        // Test de transition invalide (PAID -> DELIVERED directement)
        boolean transitionPossible = current.canTransitionTo(OrderStatus.DELIVERED);
        System.out.println("Transition PAID -> DELIVERED valide ? " + transitionPossible); //
false

        // Passage à l'état final
        current = OrderStatus.CANCELLED;
        System.out.println("Commande annulée. Est-elle fermée ? " + current.isTerminal()); //
true
    }
}

```

Chapitre 3 : Les Records (**record**)

3.1 Pourquoi les Records ? Le problème des classes DTO classique

En Java traditionnel, la création d'un objet simple de transfert de données (*Data Transfer Object* - DTO) nécessitait une quantité énorme de code verbeux (*boilerplate*) :

```

Java
// ✗ Classe DTO Classique (Plus de 40 lignes de code !)
public class PersonneDTO {
    private final Long id;
    private final String nom;
    private final String email;

    public PersonneDTO(Long id, String nom, String email) {
        this.id = id;
        this.nom = nom;
        this.email = email;
    }

    public Long getId() { return id; }
    public String getNom() { return nom; }
    public String getEmail() { return email; }

    @Override
    public boolean equals(Object o) { ... }
    @Override
    public int hashCode() { ... }
    @Override
    public String toString() { ... }
}

```

3.2 Syntaxe et fonctionnement des Records

Introduit de manière stable en **Java 16**, le **record** simplifie cela à l'extrême :

Java

//  La même structure en UNE seule ligne !

```
public record PersonneDTO(Long id, String nom, String email) {}
```

Lorsque vous définissez un **record**, le compilateur génère automatiquement :

1. Des champs **private final** pour chaque composant.
2. Un constructeur dit **canonique** qui prend tous les paramètres.
3. Des accesseurs publics portant **le même nom que le champ** (ex: `personne.nom()` et NON `personne.getNom()`).
4. Une méthode `equals()` et `hashCode()` basée sur les valeurs des champs.
5. Une méthode `toString()` lisible (ex: `PersonneDTO[id=1, nom=Alice, email=alice@test.com]`).

3.3 Le Constructeur Compact & Validations

Un record permet d'ajouter des règles de validation directement au moment de la construction sans répéter la liste des paramètres :

Java

```
public record UserRecord(Long id, String username, int age) {
```

```
    // Constructeur compact (sans paramètres entre parenthèses)
```

```
    public UserRecord {
```

```
        if (username == null || username.isBlank()) {
```

```
            throw new IllegalArgumentException("Le pseudo ne peut pas être vide");
```

```
        }
```

```
        if (age < 0) {
```

```
            throw new IllegalArgumentException("L'âge doit être positif");
```

```
        }
```

```
    }
```

```
    // Méthodes métier supplémentaires autorisées
```

```
    public boolean isAdult() {
```

```
        return this.age >= 18;
```

```
    }
```

```
}
```

3.4 Les Limites d'un Record

Bien qu'extrêmement pratiques, les records imposent des règles strictes :

1. **Immutabilité stricte** : Tous les champs sont `final`. Il n'y a pas de `setters`. Une fois instancié, l'objet ne peut plus être modifié.
2. **Pas d'héritage de classe** : Un record hérite implicitement de `java.lang.Record`. Il ne peut donc pas étendre une autre classe (`extends MaClasse` est interdit).
3. **Interfaces autorisées** : Un record peut implémenter une ou plusieurs interfaces (`implements MonInterface`).
4. **Pas d'attribut d'instance supplémentaire** : Vous ne pouvez pas ajouter d'attribut d'instance non déclaré dans l'entête du record (seuls les attributs `static` sont autorisés).



TP 2 : Traitement d'un Catalogue de Produits (*ProductRecord*)

Objectif :

Modéliser un catalogue de produits avec gestion des prix, du stock et des remises grâce aux `record`.

Consignes du TP :

1. Déclarer le record `Product(String ref, String title, double price, int stock)` avec les règles suivantes dans son constructeur compact :
 - La référence `ref` et le titre `title` ne doivent pas être nulls ni vides.
 - Le prix `price` doit être strictement supérieur à 0.
 - Le stock `stock` ne peut pas être négatif.
2. Ajouter les méthodes métier suivantes au record :
 - `boolean isInStock()` : renvoie `true` si `stock > 0`.
 - `Product withDiscount(double percentage)` : renvoie **un nouveau Record** avec le prix réduit selon le pourcentage fourni (ex: 20.0 pour 20% de remise). *Rappel : le record étant immutable, la méthode doit retourner une nouvelle instance.*
3. Tester la création, les levées d'exceptions sur données invalides et le calcul de remise dans un `Main`.



Correction détaillée du TP 2

Java

```
public record Product(String ref, String title, double price, int stock) {
```

```
    // Constructeur compact pour validation
```

```
    public Product {
```

```
        if (ref == null || ref.isBlank()) {
```

```
            throw new IllegalArgumentException("La référence produit est obligatoire.");
```

```
        }
```

```
        if (title == null || title.isBlank()) {
```

```
            throw new IllegalArgumentException("Le titre du produit est obligatoire.");
```

```
        }
```

```
        if (price <= 0) {
```

```

        throw new IllegalArgumentException("Le prix doit être supérieur à 0.");
    }
    if (stock < 0) {
        throw new IllegalArgumentException("Le stock ne peut pas être négatif.");
    }
}

// Méthode de vérification de disponibilité
public boolean isInStock() {
    return this.stock > 0;
}

// Création d'une nouvelle instance avec remise (Respect de l'immutabilité)
public Product withDiscount(double percentage) {
    if (percentage < 0 || percentage > 100) {
        throw new IllegalArgumentException("Le pourcentage de remise doit être entre 0 et 100.");
    }
    double newPrice = this.price * (1 - (percentage / 100.0));
    return new Product(this.ref, this.title, newPrice, this.stock);
}
}

```

Classe de test (Main.java) :

```

Java
public class MainTP2 {
    public static void main(String[] args) {
        // 1. Instanciation valide
        Product p1 = new Product("REF-001", "Clavier Mécanique", 89.99, 15);
        System.out.println("Produit créé : " + p1);
        System.out.println("En stock ? " + p1.isInStock());

        // 2. Application d'une remise de 20%
        Product p1Solde = p1.withDiscount(20.0);
        System.out.println("Produit soldé : " + p1Solde.title() + " -> Nouveau prix : " + p1Solde.price() + " €");

        // 3. Test de validation (Prix négatif -> doit lever une exception)
        try {
            Product p1Invalide = new Product("REF-002", "Ecran 4K", -150.0, 5);
        } catch (IllegalArgumentException e) {
            System.err.println("Erreur interceptée avec succès : " + e.getMessage());
        }
    }
}

```

Module 2 (Simplifié) : Les Génériques (Generics)

Ce module est désormais centré sur l'essentiel :

1. **Chapitre 1 : Les fondements des Génériques** (Problématique, syntaxe, sécurité de typage à la compilation)
2. **Chapitre 2 : Classes et Interfaces Génériques** (Paramètres de type **T**, **K**, **V** et conteneurs génériques)

Chapitre 1 : Les fondements des Génériques

1.1 Le problème avant Java 5 (L'époque d'Object)

Avant l'arrivée des Génériques dans Java 5, les structures de données manipulaient exclusivement la classe parente **Object**. Pour stocker divers éléments dans une liste, on écrivait :

```
Java
// ❌ Ancien Java (Sans génériques)
List maListe = new ArrayList();
maListe.add("Bonjour");
maListe.add(42); // Accepté car tout hérite d'Object

// Récupération obligatoire avec cast explicite :
String str = (String) maListe.get(0);
String str2 = (String) maListe.get(1); // 💥 ClassCastException à l'exécution !
```

Cette approche présentait deux faiblesses critiques :

1. **Absence de sécurité au typage (*Type Safety*)** : Le compilateur acceptait n'importe quel type d'objet dans la même collection.
2. **Erreurs au Runtime** : Les erreurs de type n'apparaissaient pas lors de la compilation, mais faisaient planter l'application en production avec une **ClassCastException**.

1.2 La solution : Le typage fort à la compilation

Les Génériques permettent de **spécifier le type** des données qu'une classe ou une liste accepte. Le compilateur effectue la vérification avant même d'exécuter le code.

```
Java
// ✅ Java Moderne (Avec génériques)
List<String> maListe = new ArrayList<>();
```

```
maListe.add("Bonjour");  
// maListe.add(42); //  Erreur détectée IMMÉDIATEMENT par le compilateur !  
  
String str = maListe.get(0); // Pas besoin de cast (String)
```

Chapitre 2 : Classes et Interfaces Génériques

2.1 Conventions de nommage des paramètres de type

Par convention en Java, on utilise une lettre majuscule simple pour désigner le type variable :

- **T** : Type général (Type).
- **E** : Élément (Element - utilisé dans les collections `List<E>`, `Set<E>`).
- **K** : Clé (Key - utilisé dans les `Map<K, V>`).
- **V** : Valeur (Value - utilisé dans les `Map<K, V>`).

2.2 Créer une Interface et une Classe Générique

Voici comment concevoir un conteneur générique qui peut stocker et restituer un objet de n'importe quel type unique :

```
Java  
// 1. Interface Générique avec le paramètre T  
public interface Storage<T> {  
    void store(T item);  
    T retrieve();  
}  
  
// 2. Classe qui implémente l'interface générique  
public class Box<T> implements Storage<T> {  
    private T item;  
  
    @Override  
    public void store(T item) {  
        this.item = item;  
    }  
  
    @Override  
    public T retrieve() {  
        return this.item;  
    }  
}
```

Utilisation dans le code :

Java

```
public class MainStorage {  
    public static void main(String[] args) {  
        // Une boîte qui ne contient QUE des chaînes  
        Box<String> stringBox = new Box<>();  
        stringBox.store("Document PDF");  
        String doc = stringBox.retrieve(); // Typage garanti  
  
        // Une boîte qui ne contient QUE des entiers  
        Box<Integer> intBox = new Box<>();  
        intBox.store(2026);  
        Integer annee = intBox.retrieve();  
    }  
}
```

TP Module 2 : Wrapper d'API Générique (ApiResponse<T>)

Objectif :

Créer une classe générique `ApiResponse<T>` permettant d'envelopper n'importe quel type de donnée (une String, un Record, un Objet métier) renvoyé par un service backend, avec son statut et son message d'erreur.

Consignes du TP :

1. Créer la classe générique `ApiResponse<T>` avec les attributs privés :
 - `T data` (la donnée retournée)
 - `int statusCode` (ex: 200 pour le succès, 404 pour non trouvé)
 - `String errorMessage` (nul si pas d'erreur)
2. Ajouter un constructeur qui prend ces 3 paramètres.
3. Ajouter deux constructeurs ou méthodes utilitaires simplifiées :
 - Une pour les **succès** (prend `data` et fixe `statusCode = 200`).
 - Une pour les **erreurs** (prend `errorMessage` et `statusCode`, et met `data = null`).
4. Ajouter une méthode `boolean isSuccess()` qui retourne `true` si le code HTTP est égal à `200`.
5. Dans une classe `Main`, tester `ApiResponse` avec :
 - Une donnée simple de type `String`.
 - Une donnée complexe sous forme de `record ProductRecord(String ref, double price)`.
 - Une réponse d'erreur (ex: code 404).

Correction détaillée du TP Module 2

1. La classe générique **ApiResponse<T>**

Java

```
public class ApiResponse<T> {
    private final T data;
    private final int statusCode;
    private final String errorMessage;

    // Constructeur complet
    public ApiResponse(T data, int statusCode, String errorMessage) {
        this.data = data;
        this.statusCode = statusCode;
        this.errorMessage = errorMessage;
    }

    // Constructeur pratique pour les réponses de SUCCÈS (code 200 par défaut)
    public ApiResponse(T data) {
        this(data, 200, null);
    }

    // Constructeur pratique pour les réponses d'ERREUR (sans donnée)
    public ApiResponse(String errorMessage, int statusCode) {
        this(null, statusCode, errorMessage);
    }

    public boolean isSuccess() {
        return this.statusCode == 200;
    }

    public T getData() {
        return data;
    }

    public int getStatusCode() {
        return statusCode;
    }

    public String getErrorMessage() {
        return errorMessage;
    }

    @Override
    public String toString() {
        if (isSuccess()) {
            return "ApiResponse [SUCCÈS 200] Donnée = " + data;
        } else {
            return "ApiResponse [ERREUR " + statusCode + "] Message = " + errorMessage;
        }
    }
}
```

```

        return "ApiResponse [ERREUR " + statusCode + "] Message = " + errorMessage +
        "",
    }
}
}

```

2. Classe de test (MainTP2.java)

Java

```

public class MainTP2 {

    // Record simple pour le test
    public record Product(String ref, double price) {}

    public static void main(String[] args) {

        // --- Test 1 : ApiResponse enveloppant une Simple String ---
        ApiResponse<String> stringResponse = new ApiResponse<>("Opération effectuée
avec succès !");
        System.out.println("Test 1 :");
        System.out.println("Est-ce un succès ? " + stringResponse.isSuccess());
        System.out.println("Contenu : " + stringResponse.getData());
        System.out.println(stringResponse);
        System.out.println();

        // --- Test 2 : ApiResponse enveloppant un Record Product ---
        Product p = new Product("REF-889", 49.99);
        ApiResponse<Product> productResponse = new ApiResponse<>(p);

        System.out.println("Test 2 :");
        System.out.println("Prix du produit reçu : " + productResponse.getData().price() + " €");
        System.out.println(productResponse);
        System.out.println();

        // --- Test 3 : ApiResponse en cas d'erreur ---
        ApiResponse<Product> errorResponse = new ApiResponse<>("Produit introuvable en
base", 404);

        System.out.println("Test 3 :");
        System.out.println("Est-ce un succès ? " + errorResponse.isSuccess()); // false
        System.out.println("Message d'erreur : " + errorResponse.getErrorMessage());
        System.out.println(errorResponse);
    }
}

```

Module 3 (Simplifié) : Les Annotations en Java

Ce module se concentre sur l'essentiel :

1. **Chapitre 1 : Qu'est-ce qu'une Annotation ?** (Rôle et fonctionnement)
2. **Chapitre 2 : Les Annotations Standards de Java** (`@Override`, `@Deprecated`, `@SuppressWarnings`, etc.)

Chapitre 1 : Qu'est-ce qu'une Annotation ?

1.1 Définition et Rôle

Une **annotation** est une métadonnée (un marquage) que l'on ajoute sur des éléments du code Java (classes, méthodes, attributs, paramètres) avec le symbole `@`.

Une annotation **ne modifie pas directement la logique métier** de votre code. Son rôle est de fournir des informations :

- **Au compilateur** : Pour détecter des erreurs à la compilation (ex: vérifier la redéfinition d'une méthode).
- **Aux outils et frameworks (Spring, Hibernate, Jackson, etc.)** : Pour configurer des comportements automatiquement (ex: valider des champs, mapper des bases de données).

Chapitre 2 : Les Annotations Standards de Java

Voici les annotations fournies nativement par le langage Java qu'il faut maîtriser au quotidien :

2.1 `@Override`

L'annotation `@Override` indique au compilateur qu'une méthode est destinée à **redéfinir une méthode** héritée d'une classe parente ou déclarée dans une interface.

Pourquoi est-elle essentielle ?

Elle évite les erreurs de frappe (*typos*). Si le nom de la méthode ou ses arguments ne correspondent pas exactement à la méthode parente, le compilateur lève une erreur de compilation immédiate au lieu de créer silencieusement une nouvelle méthode.

Java

```
public class Animal {  
    public void makeSound() {  
        System.out.println("Bruit générique");  
    }  
}
```

```
}  
}
```

```
public class Dog extends Animal {
```

```
    @Override
```

```
    public void makeSound() { //  Garantit que makeSound() existe bien dans Animal  
        System.out.println("Wouf !");
```

```
    }
```

```
    // Si vous écriviez @Override public void makeSond(),
```

```
    // le compilateur bloquerait immédiatement avec une erreur !
```

```
}
```

2.2 @Deprecated

L'annotation `@Deprecated` indique qu'une méthode, classe ou variable est **obsolète** et qu'elle risque d'être supprimée dans une future version du code.

- Le compilateur affiche un avertissement (*warning*) lors de son utilisation.
- On peut lui associer les attributs `since` (depuis quelle version) et `forRemoval` (si elle sera supprimée définitivement).

Java

```
public class Calculator {
```

```
    /**
```

```
     * @deprecated Utiliser {@link #sum(int, int)} à la place.
```

```
     */
```

```
    @Deprecated(since = "2.0", forRemoval = true)
```

```
    public int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    public int sum(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
}
```

2.3 @SuppressWarnings

L'annotation `@SuppressWarnings` demande au compilateur de **masquer certains avertissements spécifiés**, afin de ne pas polluer les journaux de compilation quand un avertissement est volontaire ou inévitable.

Valeurs courantes :

- `"unchecked"` : Masque les avertissements liés aux types bruts des génériques.
- `"deprecation"` : Masque les avertissements liés à l'utilisation de méthodes obsolètes.
- `"all"` : Masque tous les avertissements.

Java

```
public class Demo {  
  
    @SuppressWarnings("unchecked")  
    public void processRawList() {  
        // Code manipulant une liste non typée (ancien Java)  
        java.util.List rawList = new java.util.ArrayList();  
        rawList.add("Texte"); // Pas d'avertissement "unchecked" grâce à l'annotation  
    }  
}
```

2.4 @FunctionalInterface

Utilisée pour indiquer explicitement qu'une interface est une **interface fonctionnelle** (c'est-à-dire qu'elle ne possède **qu'une seule méthode abstraite**). Cela garantit que l'interface pourra être utilisée avec des expressions Lambda (Module 4).

Java

```
@FunctionalInterface  
public interface StringTransformer {  
    String transform(String input); // Une seule méthode abstraite autorisée  
}
```



TP Module 3 : Refactorisation et Utilisation des Annotations Standards

Objectif :

Sécuriser et nettoyer une classe de service en utilisant correctement les annotations standards de Java (`@Override`, `@Deprecated`, `@SuppressWarnings`, `@FunctionalInterface`).

Consignes du TP :

1. Créer une interface fonctionnelle `@FunctionalInterface` nommée `Formatter` possédant une seule méthode `String format(String text)`.
2. Créer une classe `OldDataService` contenant :

- Une méthode dépréciée `@Deprecated` nommée `fetchOldData()` qui renvoie une liste non typée (raw type).
- 3. Créer une classe `ModernDataService` qui hérite d'une classe de base `BaseService` :
 - Utiliser `@Override` pour redéfinir la méthode `process()`.
 - Utiliser `@SuppressWarnings("deprecation")` pour appeler la méthode obsolète de `OldDataService` sans générer d'avertissement à la compilation.
- 4. Écrire une classe `MainTP3` pour tester l'exécution.

Correction détaillée du TP Module 3

1. L'interface fonctionnelle (`Formatter.java`)

```
Java
@FunctionalInterface
public interface Formatter {
    String format(String text);
}
```

2. La classe de base (`BaseService.java`)

```
Java
public class BaseService {
    public void process() {
        System.out.println("Traitement de base...");
    }
}
```

3. L'ancien service avec méthode obsolète (`OldDataService.java`)

```
Java
import java.util.ArrayList;
import java.util.List;

public class OldDataService {

    @Deprecated(since = "1.5", forRemoval = false)
    @SuppressWarnings("rawtypes")
    public List getLegacyData() {
        List list = new ArrayList();
        list.add("Donnée héritée 1");
        list.add("Donnée héritée 2");
        return list;
    }
}
```

4. Le service moderne (**ModernDataService.java**)

Java

```
import java.util.List;
```

```
public class ModernDataService extends BaseService {

    private final OldDataService oldDataService = new OldDataService();

    @Override
    public void process() {
        System.out.println("Exécution du traitement moderne...");
    }

    // Appel à du code obsolète tout en masquant l'avertissement du compilateur
    @SuppressWarnings("deprecation")
    public void loadDataSafely() {
        List<?> data = oldDataService.getLegacyData();
        System.out.println("Données récupérées depuis l'ancien service : " + data);
    }
}
```

5. Classe de Test (**MainTP3.java**)

Java

```
public class MainTP3 {
    public static void main(String[] args) {

        // 1. Test de l'interface fonctionnelle
        Formatter upperFormatter = text -> text.toUpperCase();
        System.out.println("Formatage : " + upperFormatter.format("java moderne"));

        // 2. Test du service moderne et des annotations
        ModernDataService service = new ModernDataService();

        // Validation du comportement redéfini (@Override)
        service.process();

        // Validation de l'appel sécurisé au code déprécié (@SuppressWarnings)
        service.loadDataSafely();
    }
}
```

Module 5 : API Date/Time (**java.time**)

Ce module est découpé en 3 chapitres essentiels :

1. **Chapitre 1 : Pourquoi une nouvelle API Date/Time ?** (Problèmes de l'ancienne API et principes du package **java.time**)
2. **Chapitre 2 : Les classes principales** (**LocalDate**, **LocalTime**, **LocalDateTime**, **ZonedDateTime**)
3. **Chapitre 3 : Formatage, calcul de durées et manipulation** (**DateTimeFormatter**, **Period**, **Duration**)

Chapitre 1 : Pourquoi une nouvelle API Date/Time ?

1.1 Les problèmes de l'ancienne API (**java.util.Date** et **Calendar**)

Avant Java 8, la gestion des dates s'appuyait sur **java.util.Date** et **java.util.Calendar**. Ces anciennes classes posaient de graves problèmes :

1. **Mutabilité (Absence de thread-safety)** : Un objet **Date** pouvait être modifié après sa création (**date.setTime(...)**), ce qui provoquait des bugs imprévisibles dans les applications multi-threads.
2. **Conception confuse** :
 - Les mois commençaient à 0 (janvier = 0, décembre = 11).
 - Les années commençaient à partir de 1900 (pour l'année 2026, il fallait passer 126).
3. **Formatage non thread-safe** : La classe de formatage **SimpleDateFormat** provoquait des erreurs silencieuses lors d'accès simultanés.

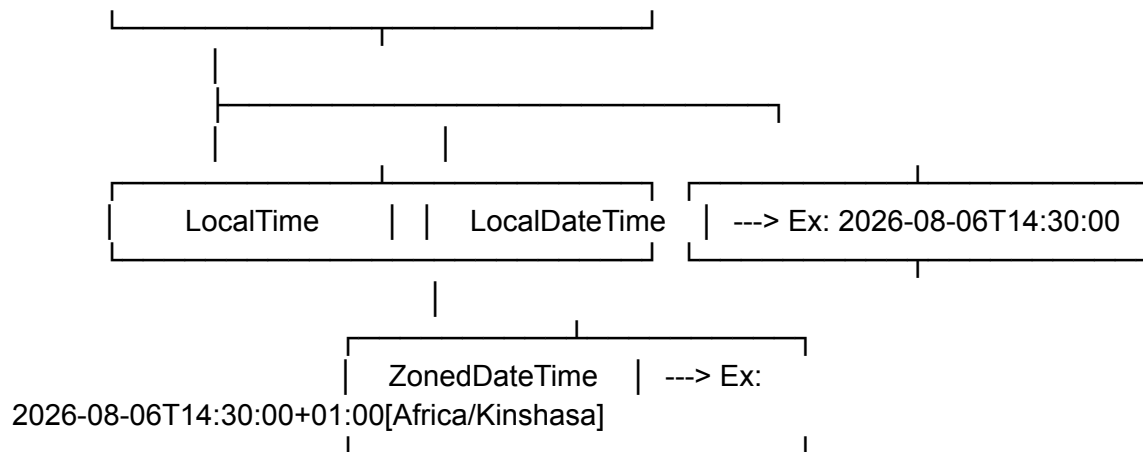
1.2 Les principes de la nouvelle API (**java.time**)

Introduite depuis Java 8, l'API issue du package **java.time** (basée sur la spécification JSR-310) repose sur 3 principes fondateurs :

- **Immutabilité stricte** : Tous les objets de date sont immutables. Toute modification (ajouter un jour, changer le mois) renvoie une **nouvelle instance**.
- **Thread-Safety native** : Les objets peuvent être partagés sans risque entre plusieurs threads.
- **Clarté des types** : Séparation nette entre une date seule, une heure seule, un horodatage avec ou sans fuseau horaire.

Chapitre 2 : Les classes principales de **java.time**

LocalDate	---> Ex: 2026-08-06 (Date seule)
-----------	----------------------------------



```
LocalDateTime startEvent = LocalDateTime.of(2026, 8, 6, 9, 0); // 06/08/2026 à 09h00
```

2.4 ZonedDateTime

Combine date, heure et **fuseau horaire** (**ZoneId**). Indispensable pour les applications internationales ou les calculs de décalage horaire.

Java

```
ZonedDateTime kinshasaZone = ZonedDateTime.of("Africa/Kinshasa");  
ZonedDateTime localZoned = ZonedDateTime.now(kinshasaZone);
```

```
ZonedDateTime parisZone = ZonedDateTime.of("Europe/Paris");  
ZonedDateTime parisZoned = localZoned.withZoneSameInstant(parisZone); // Convertit  
l'heure exacte vers Paris
```

Chapitre 3 : Formatage, Calculs et Durées

3.1 Formatage avec DateTimeFormatter

DateTimeFormatter remplace l'ancien **SimpleDateFormat**. Il est thread-safe et immutable.

Java

```
LocalDateTime now = LocalDateTime.now();
```

// 1. Utilisation d'un pattern personnalisé

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");  
String formattedDate = now.format(formatter); // "06/08/2026 14:30:00"
```

// 2. Conversion d'une chaîne en Date (Parsing)

```
String dateText = "15/09/2026";  
DateTimeFormatter dateOnlyFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");  
LocalDate parsedDate = LocalDate.parse(dateText, dateOnlyFormatter);
```

3.2 Calculs sur les dates (Ajustements immutables)

Grâce à l'immutabilité, les méthodes de calcul renvoient toujours un nouvel objet :

Java

```
LocalDate date = LocalDate.of(2026, 8, 6);
```

```
LocalDate nextWeek = date.plusWeeks(1); // Ajoute 1 semaine
```

```
LocalDate previousMonth = date.minusMonths(2); // Retire 2 mois
```

3.3 Period vs Duration

- **Period** : Mesure une quantité de temps en **unités de dates** (années, mois, jours).
- **Duration** : Mesure une quantité de temps en **unités d'heures/secondes** (heures, minutes, secondes, nanosecondes).

Java

// Exemple Period (Jours/Mois/Années)

LocalDate start = LocalDate.of(2026, 1, 1);

LocalDate end = LocalDate.of(2026, 8, 6);

Period period = Period.between(start, end);

System.out.println("Écart : " + period.getMonths() + " mois et " + period.getDays() + " jours");

// Exemple Duration (Heures/Minutes/Secondes)

LocalTime t1 = LocalTime.of(09, 00);

LocalTime t2 = LocalTime.of(17, 30);

Duration duration = Duration.between(t1, t2);

System.out.println("Durée du travail : " + duration.toHours() + " heures");



TP Module 5 : Calculateur d'Échéance de Facturation & Ajustement Week-end

Objectif :

Créer un composant métier capable de calculer la date d'échéance exacte d'une facture, d'ajuster l'échéance si elle tombe un week-end (report au lundi suivant), et de calculer le nombre de jours restants avant le terme.

Consignes du TP :

1. Créer une classe `InvoiceService`.
2. Écrire une méthode `public LocalDate calculateDueDate(LocalDate issueDate, int paymentTermDays)` qui :
 - Ajoute le nombre de jours du délai de paiement (`paymentTermDays`) à la date d'émission (`issueDate`).
 - Vérifie si la date d'échéance calculée tombe un **samedi** ou un **dimanche** :
 - Si c'est un samedi : ajouter 2 jours (reporter au lundi).
 - Si c'est un dimanche : ajouter 1 jour (reporter au lundi).
 - Retourne la date ajustée.
3. Écrire une méthode `public String formatInvoiceDetails(LocalDate issueDate, LocalDate dueDate)` qui :
 - Formate la date d'émission et la date d'échéance selon le format français `"dd/MM/yyyy"`.
 - Calcule le nombre de jours exacts entre la date d'émission et l'échéance finale à l'aide de `Period` ou `ChronoUnit.DAYS`.
 - Renvoie une chaîne descriptive lisible.

4. Tester dans un **MainTP5** avec deux cas de figure :
- Un cas où l'échéance tombe un jour ouvré (en semaine).
 - Un cas où l'échéance tombe un samedi ou dimanche (pour vérifier l'ajustement au lundi).

Correction détaillée du TP Module 5

1. Le Service Métier (**InvoiceService.java**)

Java

```
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class InvoiceService {

    private static final DateTimeFormatter DATE_FORMATTER =
        DateTimeFormatter.ofPattern("dd/MM/yyyy");

    /**
     * Calcule la date d'échéance d'une facture avec ajustement si week-end.
     */
    public LocalDate calculateDueDate(LocalDate issueDate, int paymentTermDays) {
        if (issueDate == null) {
            throw new IllegalArgumentException("La date d'émission ne peut pas être nulle.");
        }

        // 1. Calcul initial de la date d'échéance
        LocalDate targetDate = issueDate.plusDays(paymentTermDays);

        // 2. Vérification et ajustement si week-end
        DayOfWeek dayOfWeek = targetDate.getDayOfWeek();

        if (dayOfWeek == DayOfWeek.SATURDAY) {
            targetDate = targetDate.plusDays(2); // Samedi -> Lundi
        } else if (dayOfWeek == DayOfWeek.SUNDAY) {
            targetDate = targetDate.plusDays(1); // Dimanche -> Lundi
        }

        return targetDate;
    }

    /**
     * Génère un résumé textuel de la facture.
     */
    public String formatInvoiceDetails(LocalDate issueDate, LocalDate dueDate) {
        String formattedIssue = issueDate.format(DATE_FORMATTER);
```

```

String formattedDue = dueDate.format(DATE_FORMATTER);

long totalDays = ChronoUnit.DAYS.between(issueDate, dueDate);

return String.format("Facture émise le : %s | Échéance : %s (Délai réel : %d jours)",
    formattedIssue, formattedDue, totalDays);
}
}

```

2. Classe de Test (MainTP5.java)

Java

```
import java.time.LocalDate;
```

```

public class MainTP5 {
    public static void main(String[] args) {
        InvoiceService invoiceService = new InvoiceService();

        // --- Cas 1 : Échéance qui tombe en semaine ---
        // Exemple : 01/08/2026 + 10 jours = 11/08/2026 (Mardi)
        LocalDate issueDate1 = LocalDate.of(2026, 8, 1);
        LocalDate dueDate1 = invoiceService.calculateDueDate(issueDate1, 10);

        System.out.println("--- Cas 1 (Semaine) ---");
        System.out.println("Jour d'échéance : " + dueDate1.getDayOfWeek()); // TUESDAY
        System.out.println(invoiceService.formatInvoiceDetails(issueDate1, dueDate1));
        System.out.println();

        // --- Cas 2 : Échéance qui tombe un Samedi ---
        // Exemple : 05/08/2026 + 30 jours = 04/09/2026 -> 05/09/2026 (Samedi) -> Ajusté au
        Lundi 07/09/2026
        LocalDate issueDate2 = LocalDate.of(2026, 8, 6);
        LocalDate dueDate2 = invoiceService.calculateDueDate(issueDate2, 30); // 5
        Septembre 2026 est un Samedi

        System.out.println("--- Cas 2 (Ajustement Week-end) ---");
        System.out.println("Jour d'échéance ajusté : " + dueDate2.getDayOfWeek()); //
        MONDAY
        System.out.println(invoiceService.formatInvoiceDetails(issueDate2, dueDate2));
    }
}

```

Module 6 : Le Type `Optional<T>`

Ce module est découpé en 3 chapitres pratiques :

1. Chapitre 1 : Le problème du `null` et le rôle de `Optional`
2. Chapitre 2 : Créer et manipuler un `Optional`
3. Chapitre 3 : Extraire les valeurs et enchaîner proprement

Chapitre 1 : Le problème du `null` et le rôle de `Optional`

1.1 La "Billion Dollar Mistake"

Déclarer ou retourner `null` pour signifier l'absence d'une valeur est la cause de la très célèbre `NullPointerException` (NPE).

Java

```
// ❌ Risque constant de NullPointerException
public Customer findCustomerById(String id) {
    if (!exists(id)) {
        return null; // L'appelant doit absolument penser à vérifier si la valeur est null
    }
    return new Customer(id, "Alice");
}
```

// Code appelant fébrile :

```
Customer c = findCustomerById("99");
System.out.println(c.getName().toUpperCase()); // 💣 NPE si c est null !
```

1.2 La solution : `Optional<T>`

Introduit en Java 8, `Optional<T>` est un **conteneur unique** qui contient soit une valeur non nulle de type `T`, soit rien du tout (*empty*).

- Il rend le contrat d'une méthode explicite : si une méthode retourne `Optional<Customer>`, le développeur **sait immédiatement** que la valeur peut être absente.
- Il force à traiter l'absence de valeur de façon propre, sans multiplier les blocs `if (obj != null)`.

Chapitre 2 : Créer et Manipuler un `Optional`

2.1 Les 3 manières de créer un **Optional**

Java

// 1. Un Optional vide (absence de valeur)

```
Optional<String> emptyOpt = Optional.empty();
```

// 2. Un Optional avec une valeur NON NULLE (lève une NPE immédiatement si la valeur est null)

```
Optional<String> valueOpt = Optional.of("Java");
```

// 3. Un Optional sécurisé (accepte une valeur ou null sans planter)

```
String name = getNullableName();
```

```
Optional<String> safeOpt = Optional.ofNullable(name); // Soit contient "name", soit  
Optional.empty()
```

2.2 Vérifier la présence d'une valeur

Java

```
Optional<String> opt = Optional.ofNullable(getName());
```

// Vérification classique

```
if (opt.isPresent()) {  
    System.out.println("Valeur trouvée : " + opt.get());  
}
```

// Vérification de l'absence

```
if (opt.isEmpty()) {  
    System.out.println("Aucune valeur présente.");  
}
```

// Exécuter une action SEULEMENT si la valeur est présente (style fonctionnel)

```
opt.ifPresent(val -> System.out.println("Bonjour " + val));
```

Chapitre 3 : Extraire les valeurs et enchaîner proprement

Au lieu d'utiliser `isPresent()` + `get()`, la puissance d'Optional réside dans ses méthodes d'extraction sécurisées.

3.1 Fournir une valeur de repli (*Fallback*)

Java

```
Optional<String> opt = Optional.ofNullable(maybeNullValue);
```

// 1. `orElse` : retourne la valeur ou une valeur par défaut

```
String result1 = opt.orElse("Valeur par défaut");
```

```
// 2. orElseGet : calcule la valeur par défaut à la demande (paresseuse/lazy)
String result2 = opt.orElseGet(() -> generateDefaultValue());

// 3. orElseThrow : lève une exception sur mesure si la valeur est absente
String result3 = opt.orElseThrow(() -> new IllegalArgumentException("Donnée obligatoire manquante !"));
```

3.2 Transformer et filtrer un **Optional**

On peut appliquer des transformations directement sur un **Optional** sans risque de NPE :

```
Java
Optional<String> nameOpt = Optional.ofNullable("michee");

// map() : transforme la valeur si elle existe
Optional<String> upperOpt = nameOpt.map(String::toUpperCase); // Optional["MICHEE"]

// filter() : conserve la valeur seulement si elle respecte une condition
Optional<String> longNameOpt = nameOpt.filter(name -> name.length() > 10); //
Optional.empty()
```



TP Module 6 : Service de Recherche Utilisateur & Gestion de Profil

Objectif :

Créer un service de recherche d'utilisateurs sécurisé avec **Optional**, permettant de récupérer le nom en majuscule d'un utilisateur ou de lever une exception métier claire.

Consignes du TP :

1. Créer un **record User(String id, String name, String email)**.
2. Créer un composant **UserRepository** contenant une liste d'utilisateurs enregistrés en mémoire.
3. Écrire la méthode **public Optional<User> findById(String id)** dans le repository.
4. Créer la classe **UserService** utilisant **UserRepository** et écrire les méthodes suivantes :
 - **public String getUsernameUpper(String id)** : Cherche l'utilisateur par ID, extrait son nom, le convertit en majuscule, ou retourne **"INVITÉ"** si non trouvé.

- `public User getUserOrThrow(String id)` : Recherche l'utilisateur et retourne l'objet `User`, ou lève une exception `RuntimeException("Utilisateur non trouvé avec l'ID : " + id)`.

5. Tester dans un `MainTP6` avec un utilisateur existant et un utilisateur inexistant.

Correction détaillée du TP Module 6

1. Le modèle (`User.java`)

Java

```
public record User(String id, String name, String email) {}
```

2. Le Repository de données (`UserRepository.java`)

Java

```
import java.util.List;
```

```
import java.util.Optional;
```

```
public class UserRepository {
```

```
    private final List<User> users = List.of(
        new User("USR-101", "Michée Ndongala", "michee@example.com"),
        new User("USR-102", "Sarah Connor", "sarah@example.com")
    );
```

```
    /**
```

```
     * Recherche un utilisateur par ID et retourne un Optional.
```

```
    */
```

```
    public Optional<User> findById(String id) {
        if (id == null) {
            return Optional.empty();
        }
```

```
        for (User u : users) {
            if (u.id().equalsIgnoreCase(id)) {
                return Optional.of(u);
            }
        }
```

```
        return Optional.empty();
    }
}
```

3. Le Service Métier (`UserService.java`)

Java

```

public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    /**
     * Récupère le nom en majuscules ou retourne "INVITÉ" de manière fluide.
     */
    public String getUsernameUpper(String id) {
        return userRepository.findById(id)
            .map(User::name)
            .map(String::toUpperCase)
            .orElse("INVITÉ");
    }

    /**
     * Récupère l'utilisateur ou lève une exception claire.
     */
    public User getUserOrThrow(String id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Utilisateur introuvable avec l'ID
: " + id));
    }
}

```

4. Classe de Test (MainTP6.java)

Java

```

public class MainTP6 {
    public static void main(String[] args) {
        UserRepository repository = new UserRepository();
        UserService service = new UserService(repository);

        System.out.println("--- Test 1 : Recherche d'un utilisateur existant ---");
        String name1 = service.getUsernameUpper("USR-101");
        System.out.println("Nom récupéré : " + name1); // "MICHÉE NDONGALA"

        User user1 = service.getUserOrThrow("USR-101");
        System.out.println("Utilisateur trouvé : " + user1);

        System.out.println();

        System.out.println("--- Test 2 : Recherche d'un utilisateur inexistant ---");
        String name2 = service.getUsernameUpper("USR-999");
    }
}

```

```

        System.out.println("Nom récupéré : " + name2); // "INVITÉ"

        try {
            service.getUserOrThrow("USR-999");
        } catch (IllegalArgumentException e) {
            System.err.println("Exception interceptée comme prévu : " + e.getMessage());
        }
    }
}

```

Module 4 : Expressions Lambda & Stream API

Ce module se divise en 3 chapitres complémentaires :

1. **Chapitre 1 : Les Expressions Lambda** (Syntaxe et interfaces fonctionnelles)
2. **Chapitre 2 : Les Références de Méthodes** (Syntaxe compacte `Class::method`)
3. **Chapitre 3 : La Stream API** (Filtrage, transformation et agrégation de données)

Chapitre 1 : Les Expressions Lambda

1.1 De la classe anonyme à la Lambda

Avant Java 8, pour passer un comportement en paramètre (par exemple pour trier une liste), il fallait écrire une **classe anonyme**, souvent verbeuse :

```

Java
// ❌ Ancien Java : Classe anonyme
Comparator<String> comp = new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.compareTo(s2);
    }
};

```

Une **expression Lambda** permet d'écrire cette même logique sous forme de fonction anonyme concise :

```

Java
// ✅ Java Moderne : Expression Lambda
Comparator<String> comp = (s1, s2) -> s1.compareTo(s2);

```

1.2 Syntaxe d'une Lambda

Une lambda se compose de trois parties :

1. **Les paramètres** : `(a, b)`
2. **La flèche** : `->`

3. **Le corps** : Une expression simple ou un bloc d'instructions { ... }.

```
Java
// Sans paramètre
Runnable r = () -> System.out.println("Hello");

// Un seul paramètre (les parenthèses sont optionnelles)
Consumer<String> printer = s -> System.out.println(s);

// Plusieurs paramètres avec bloc de code
BiFunction<Integer, Integer, Integer> adder = (a, b) -> {
    int sum = a + b;
    return sum;
};
```

1.3 Les Interfaces Fonctionnelles Clés (**java.util.function**)

Une lambda s'appuie toujours sur une **interface fonctionnelle** (une interface n'ayant qu'une seule méthode abstraite). Java fournit un ensemble d'interfaces prêtes à l'emploi :

Interface	Signature	Rôle	Exemple
Predicate<T>	boolean test(T t)	Vérifie une condition (retourne vrai/faux)	s -> s.isEmpty()
Function<T, R>	R apply(T t)	Transforme une donnée de type T en type R	s -> s.length()
Consumer<T>	void accept(T t)	Consomme une donnée sans rien retourner	s -> System.out.println(s)
Supplier<T>	T get()	Fournit une valeur sans paramètre en entrée	() -> LocalDate.now()

Chapitre 2 : Les Références de Méthodes (::)

Quand une expression Lambda se contente d'appeler une méthode existante sans transformation, on peut la raccourcir grâce aux **références de méthodes** (::).

```

Java
// Lambda classique :
Consumer<String> c1 = s -> System.out.println(s);
// Référence de méthode équivalente :
Consumer<String> c2 = System.out::println;

// Lambda classique :
Function<String, Integer> f1 = s -> Integer.parseInt(s);
// Référence de méthode équivalente :
Function<String, Integer> f2 = Integer::parseInt;

```

Chapitre 3 : La Stream API (**java.util.stream**)

3.1 Qu'est-ce qu'un **Stream** ?

Un **Stream** est une séquence d'éléments permettant d'effectuer des traitements de données déclaratifs (filtrage, tri, transformation) de façon fluide.

Un traitement sur un Stream suit toujours un schéma en **3 étapes** :

Source (Collection, Array) —> Opérations Intermédiaires (filter, map...) —> Opération Terminale (collect, count...)

3.2 Les Opérations Intermédiaires (Retournent un nouveau Stream)

Ces opérations sont exécutées de manière **paresseuse** (*lazy*) : elles ne s'exécutent que lorsqu'une opération terminale est appelée.

- **filter(Predicate<T>)** : Conserve uniquement les éléments respectant la condition.
- **map(Function<T, R>)** : Transforme chaque élément du Stream.
- **sorted()** / **sorted(Comparator)** : Trie les éléments.
- **distinct()** : Élimine les doublons.
- **limit(n)** : Ne conserve que les \$n\$ premiers éléments.

3.3 Les Opérations Terminales (Déclenchent le calcul et mettent fin au Stream)

- **collect(Collectors.toList())** / **.toList()** : Rassemble les résultats dans une liste.
- **forEach(Consumer<T>)** : Parcourt chaque élément.
- **count()** : Compte le nombre d'éléments.
- **findFirst()** / **findAny()** : Récupère le premier élément trouvé (retourne un **Optional**).
- **anyMatch(Predicate)** / **allMatch(Predicate)** : Renvoie **true** si la condition est remplie.

```

Java
List<String> names = List.of("Alice", "Bob", "Alexandre", "Charlie");

List<String> result = names.stream()
    .filter(name -> name.startsWith("A")) // Filtre les noms commençant par "A"
    .map(String::toUpperCase)           // Convertit en majuscules

```

```
.sorted()           // Trie par ordre alphabétique  
.toList();         // Collecte dans une nouvelle liste
```

```
System.out.println(result); // [ALEXANDRE, ALICE]
```

TP Module 4 : Traitement et Analyse d'un Catalogue de Produits

Objectif :

Manipuler une liste de produits en mémoire en combinant les Lambdas, les Références de méthodes et la Stream API pour effectuer des filtrages, des calculs et des transformations.

Consignes du TP :

1. Créer un **record Product**(String name, String category, double price, boolean available).
2. Dans une classe **ProductService** ou un **MainTP4**, créer une liste de produits d'exemple :
 - Laptop (Électronique, 1200.0, disponible)
 - Smartphone (Électronique, 800.0, disponible)
 - Souris (Électronique, 25.0, indisponible)
 - Chaise de bureau (Mobilier, 150.0, disponible)
 - Table (Mobilier, 300.0, disponible)
3. **Réaliser les 4 opérations suivantes via la Stream API :**
 - **Opération 1 :** Récupérer la liste des noms des produits **disponibles**, convertis en **majuscules**, triés par ordre alphabétique.
 - **Opération 2 :** Calculer le **prix total** de tous les produits de la catégorie "**Électronique**".
 - **Opération 3 :** Trouver le produit **le moins cher** parmi les produits disponibles.
 - **Opération 4 :** Vérifier si tous les produits du catalogue ont un prix supérieur à **10.0 € / \$**.

Correction détaillée du TP Module 4

1. Le Record Produit (**Product.java**)

Java

```
public record Product(String name, String category, double price, boolean available) {}
```

2. La classe de traitement (**MainTP4.java**)

Java

```
import java.util.Comparator;  
import java.util.List;  
import java.util.Optional;
```

```
public class MainTP4 {  
    public static void main(String[] args) {
```

```

List<Product> catalog = List.of(
    new Product("Laptop", "Électronique", 1200.0, true),
    new Product("Smartphone", "Électronique", 800.0, true),
    new Product("Souris", "Électronique", 25.0, false),
    new Product("Chaise de bureau", "Mobilier", 150.0, true),
    new Product("Table", "Mobilier", 300.0, true)
);

// --- Opération 1 : Noms des produits disponibles en majuscules et triés ---
List<String> availableProductNames = catalog.stream()
    .filter(Product::available)          // Garde seulement disponibles (available == true)
    .map(Product::name)                 // Extrait le nom
    .map(String::toUpperCase)           // Convertit en majuscule
    .sorted()                           // Trie
    .toList();                           // Collecte

System.out.println("1. Produits disponibles triés : " + availableProductNames);

// --- Opération 2 : Prix total des produits de la catégorie "Électronique" ---
double totalPriceElectronics = catalog.stream()
    .filter(p -> p.category().equalsIgnoreCase("Électronique"))
    .mapToDouble(Product::price)        // Transforme en DoubleStream pour le calcul
    .sum();

// --- Opération 3 : Produit disponible le moins cher ---
Optional<Product> cheapestProduct = catalog.stream()
    .filter(Product::available)
    .min(Comparator.comparingDouble(Product::price));

cheapestProduct.ifPresent(p ->
    System.out.println("3. Produit disponible le moins cher : " + p.name() + " (" + p.price()
+ " $)"));

// --- Opération 4 : Vérification si tous les produits sont > 10.0 ---
boolean allAboveTen = catalog.stream()
    .allMatch(p -> p.price() > 10.0);

System.out.println("4. Tous les produits coûtent-ils plus de 10 $ ? " + allAboveTen);
}
}

```

Module 7 : Apache Maven (Gestion de projet & Dépendances)

Ce module est découpé en 3 chapitres essentiels :

1. **Chapitre 1 : Qu'est-ce que Maven et le fichier `pom.xml` ?**
2. **Chapitre 2 : Gestion des Dépendances et Repositories**
3. **Chapitre 3 : Le Cycle de Vie d'un Build Maven (*Build Lifecycle*)**

Chapitre 1 : Qu'est-ce que Maven et le fichier `pom.xml` ?

1.1 À quoi sert Maven ?

En Java, dès que l'on veut ajouter des bibliothèques externes (drivers de base de données, frameworks, outils JSON), gérer la compilation ou packager son application en fichier `.jar`, le faire à la main devint ingérable.

Maven est un outil d'automatisation de build et de gestion de dépendances. Il apporte :

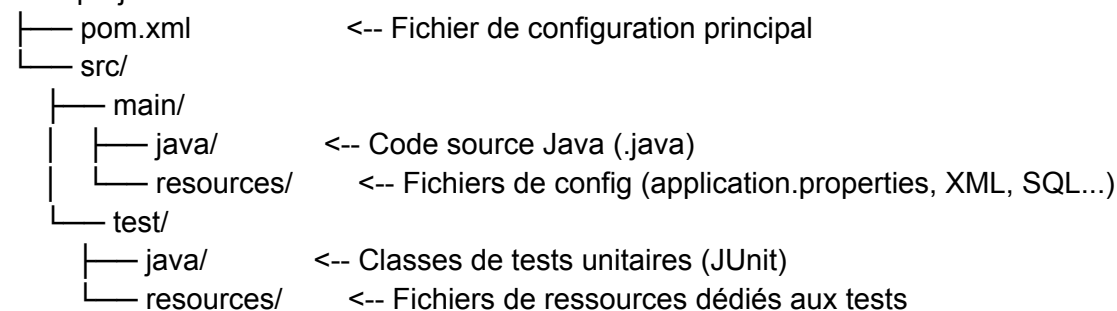
- Une **structure de projet standardisée**.
- Une gestion automatique des dépendances (téléchargement des fichiers `.jar` et de leurs propres sous-dépendances).
- Un **cycle de vie de build unifié** (compile, test, package, deploy).

1.2 Structure standard d'un projet Maven

Maven impose une convention d'organisation des dossiers (*Convention over Configuration*) :

Plaintext

mon-projet/



1.3 Le fichier `pom.xml` (Project Object Model)

Le fichier `pom.xml` situé à la racine est la carte d'identité du projet.

Coordonnées de base (GAV : Group, Artifact, Version) :

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                        http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<!-- 1. Identifiant unique de l'organisation ou package principal -->
<groupId>com.monentreprise.app</groupId>

<!-- 2. Nom du projet / composant (nom du fichier généré) -->
<artifactId>mon-application</artifactId>

<!-- 3. Version de votre projet -->
<version>1.0.0-SNAPSHOT</version>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
</project>

```

Remarque sur -SNAPSHOT : Signale une version en cours de développement.
 Quand la version est stable et finale, on retire le suffixe -SNAPSHOT (ex: 1.0.0).

Chapitre 2 : Gestion des Dépendances & Repositories

2.1 Ajouter une dépendance

Pour ajouter une bibliothèque externe (ex: **Jackson** pour manipuler du JSON ou **JUnit** pour les tests), on l'ajoute dans la section `<dependencies>` du `pom.xml`.

XML

```

<dependencies>
  <!-- Dépendance pour manipuler du JSON avec Jackson -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
  </dependency>

  <!-- Dépendance pour les tests unitaires -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

```

2.2 La portée d'une dépendance (`<scope>`)

La balise `<scope>` indique à Maven quand la dépendance doit être disponible :

- **compile (Par défaut)** : Disponible partout (compilation, tests, exécution finale).
- **test** : Disponible uniquement lors de la phase de compilation et d'exécution des tests (ex: JUnit). Elle ne sera pas incluse dans l'application livrée en production.
- **provided** : Requise pour la compilation, mais fournie au Runtime par le serveur d'application (ex: Servlet API fourni par Tomcat).
- **runtime** : Non requise pour la compilation, mais requise pour l'exécution (ex: Driver JDBC MySQL/PostgreSQL).

2.3 Repositories Local vs Remote

Plaintext

[Maven Central] (Dépôt officiel sur internet)

|

▼ (Téléchargement automatique)

[Local Repository] (~/.m2/repository sur votre machine)

|

▼ (Inclusion dans le projet)

[Votre Projet Java]

1. Maven vérifie d'abord si le fichier `.jar` existe dans votre **Repository Local** (sur votre machine, dans le dossier `~/.m2/repository`).
2. S'il ne s'y trouve pas, il le télécharge automatiquement depuis **Maven Central** (ou un dépôt d'entreprise privé comme Nexus/JFrog) et le met en cache localement.

Chapitre 3 : Le Cycle de Vie d'un Build Maven (Lifecycle)

Un cycle de vie Maven est composé d'une suite de **phases**. Exécuter une phase déclenche automatiquement toutes les phases précédentes.

Les phases principales du cycle **default** :

Commande CLI	Phase	Action
<code>mvn validate</code>	Validate	Vérifie que le projet est correct et toutes les infos disponibles.
<code>mvn compile</code>	Compile	Compile le code source Java (<code>src/main/java</code>) vers les fichiers <code>.class</code> dans <code>target/classes</code> .

<code>mvn test</code>	Test	Compile et exécute les tests unitaires (<code>src/test/java</code>).
<code>mvn package</code>	Package	Emballer le code compilé dans son format de distribution (ex: fichier <code>.jar</code> ou <code>.war</code>) dans le dossier <code>target/</code> .
<code>mvn verify</code>	Verify	Exécute des contrôles de qualité sur le livrable.
<code>mvn install</code>	Install	Installe le package <code>.jar</code> dans votre dépôt local (<code>~/.m2/repository</code>) pour l'utiliser dans d'autres projets locaux.

Commande de nettoyage courante :

- `mvn clean` : Supprime le dossier `target/` généré par les builds précédents.
- `mvn clean package` : Nettoie l'ancien build puis recompile et génère le nouveau `.jar`.



TP Module 7 : Configuration d'un projet Maven & Intégration de Jackson JSON

Objectif :

Configurer un projet Maven à partir d'un fichier `pom.xml`, déclarer la dépendance Jackson, et sérialiser/désérialiser un objet Java en chaîne JSON.

Consignes du TP :

1. Rédiger le fichier `pom.xml` pour un projet nommé `maven-demo` sous le groupe `com.demo` avec Java 17.
2. Ajouter la dépendance Jackson Databind :
 - `groupId` : `com.fasterxml.jackson.core`
 - `artifactId` : `jackson-databind`
 - `version` : `2.17.0`
3. Créer un record `Student(String name, int age, String course)`.
4. Écrire une classe `MainTP7` qui utilise la classe `ObjectMapper` de Jackson pour :
 - Convertir une instance de `Student` en chaîne JSON.
 - Convertir une chaîne JSON de test en objet `Student`.



Correction détaillée du TP Module 7

1. Le fichier `pom.xml`

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                           http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<groupId>com.demo</groupId>
<artifactId>maven-demo</artifactId>
<version>1.0.0-SNAPSHOT</version>

<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
    <!-- Dépendance Jackson pour la manipulation JSON -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.17.0</version>
    </dependency>
</dependencies>
</project>

```

2. Le Record (**Student.java**) dans **src/main/java/com/demo/Student.java**

Java

```
package com.demo;
```

```
public record Student(String name, int age, String course) {}
```

3. La classe principale (**MainTP7.java**) dans **src/main/java/com/demo/MainTP7.java**

Java

```
package com.demo;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

```

public class MainTP7 {
    public static void main(String[] args) {
        ObjectMapper mapper = new ObjectMapper();

        try {
            // --- 1. Sérialisation (Objet Java -> JSON) ---
            Student student = new Student("Michée", 25, "Java Avancé & Maven");
            String jsonOutput =
mapper.writerWithDefaultPrettyPrinter().writeValueAsString(student);

            System.out.println("--- Objet Java converti en JSON ---");

```

```

        System.out.println(jsonOutput);

        System.out.println();

        // --- 2. Désérialisation (JSON -> Objet Java) ---
        String jsonInput = "{\"name\":\"Sarah\",\"age\":22,\"course\":\"Architecture
Microservices\"}";
        Student parsedStudent = mapper.readValue(jsonInput, Student.class);

        System.out.println("--- JSON converti en Objet Java ---");
        System.out.println("Nom : " + parsedStudent.name());
        System.out.println("Âge : " + parsedStudent.age());
        System.out.println("Cours : " + parsedStudent.course());

    } catch (Exception e) {
        System.err.println("Erreur lors du traitement JSON : " + e.getMessage());
    }
}
}

```

1. À quoi sert un plugin ?

Les plugins permettent d'étendre et de personnaliser tout ce qui se passe durant le build.
Par exemple :

- **Compiler le code Java** : Le plugin `maven-compiler-plugin` transforme tes fichiers `.java` en `.class`.
- **Exécuter les tests** : Le plugin `maven-surefire-plugin` lance tes tests unitaires (JUnit).
- **Packager l'application** : Le plugin `maven-jar-plugin` (ou `maven-war-plugin`) assemble tes fichiers compilés dans une archive.
- **Créer un JAR exécutable ("fat-jar")** : Le plugin `maven-assembly-plugin` ou `maven-shade-plugin` embarque le projet avec toutes ses dépendances dans un seul fichier JAR.
- **Analyser le code** : Des plugins comme `spotbugs-maven-plugin` ou `jacoco-maven-plugin` vérifient la qualité du code ou la couverture de tests.

2. Dépendance vs Plugin : Quelle est la différence ?

Il ne faut pas confondre les deux sections du `pom.xml` :

Concept	Balise XML	Rôle	Exemple
Dépendance	<code><dependencies></code>	Bibliothèque de code importée pour être utilisée dans tes	Jackson, Driver JDBC, Spring

		classes (via des import).	
Plugin	<code><build><plugins></code>	Outil exécuté par Maven pendant la phase de build pour accomplir une action sur le projet.	Compilateur, Générateur de doc, Packager

3. Comment configurer un plugin dans le **pom.xml** ?

Les plugins s'ajoutent à l'intérieur du bloc `<build><plugins>`.

Voici un exemple concret avec la configuration du compilateur (**maven-compiler-plugin**) pour forcer l'utilisation de **Java 17** :

XML

```
<project>
...
<build>
  <plugins>
    <!-- Plugin pour configurer la compilation Java -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source> <!-- Version du code source Java -->
        <target>17</target> <!-- Version du bytecode cible -->
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

4. Lier un plugin à une phase du cycle de vie (**Goal Execution**)

Chaque plugin possède un ou plusieurs **Goals** (objectifs). Tu peux rattacher un goal spécifique à une phase du cycle de vie de Maven.

Exemple : Supposons qu'on veuille générer un JAR exécutable "all-in-one" (avec dépendances incluses) automatiquement lorsqu'on lance la commande **mvn package**.

XML

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-assembly-plugin</artifactId>
  <version>3.6.0</version>
  <executions>
    <execution>
      <phase>package</phase> <!-- S'exécute pendant 'mvn package' -->
      <goals>
        <goal>single</goal> <!-- Exécute le goal 'single' du plugin -->
      </goals>
    </execution>
  </executions>
  <configuration>
    <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
  </configuration>
</plugin>

```

TP Module 9 : Journalisation d'un Service Authentification & Gestion d'Erreurs

Objectif :

Implémenter un service d'authentification sécurisé qui journalise correctement chaque étape critique (succès, avertissement de sécurité, erreur système) selon le niveau approprié.

Consignes du TP :

1. Créer une classe `AuthenticationService`.
2. Initialiser le logger SLF4J pour cette classe.
3. Écrire la méthode `public boolean login(String username, String password)` :
 - Émettre un log `DEBUG` indiquant la tentative de connexion pour l'utilisateur.
 - Si `username` ou `password` est `null` ou vide, émettre un log `WARN` et retourner `false`.
 - Si les identifiants sont `"admin" / "secret123"`, émettre un log `INFO` de succès et retourner `true`.
 - Si le mot de passe est incorrect, émettre un log `WARN` avec le nom d'utilisateur et retourner `false`.
 - Simuler une exception système si `username` est `"error_trigger"`, intercepter l'exception et émettre un log `ERROR` avec la stack trace complète.
4. Écrire une classe `MainTP9` pour tester tous les scénarios de journalisation.

Correction détaillée du TP Module 9

1. Le Service d'Authentification (**AuthenticationService.java**)

Java

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
public class AuthenticationService {
```

```
    private static final Logger logger = LoggerFactory.getLogger(AuthenticationService.class);
```

```
    public boolean login(String username, String password) {
```

```
        logger.debug("Tentative d'authentification pour l'utilisateur: '{}", username);
```

```
        // Validation des paramètres
```

```
        if (username == null || username.isBlank() || password == null || password.isBlank()) {
```

```
            logger.warn("Tentative de connexion avec des identifiants vides ou nuls");
```

```
            return false;
```

```
        }
```

```
        // Simulation d'une erreur système grave
```

```
        if ("error_trigger".equalsIgnoreCase(username)) {
```

```
            try {
```

```
                throw new IllegalStateException("Service de base de données indisponible");
```

```
            } catch (Exception e) {
```

```
                logger.error("Erreur critique lors de la vérification des identifiants pour '{}",  
username, e);
```

```
                return false;
```

```
            }
```

```
        }
```

```
        // Vérification des identifiants
```

```
        if ("admin".equals(username) && "secret123".equals(password)) {
```

```
            logger.info("Authentification réussie pour l'utilisateur: '{}", username);
```

```
            return true;
```

```
        } else {
```

```
            logger.warn("Échec d'authentification: mot de passe incorrect pour l'utilisateur: '{}",  
username);
```

```
            return false;
```

```
        }
```

```
    }
```

```
}
```

2. Classe de Test (**MainTP9.java**)

Java

```
public class MainTP9 {
```

```

public static void main(String[] args) {
    AuthenticationService authService = new AuthenticationService();

    System.out.println("--- Test 1 : Connexion valide ---");
    authService.login("admin", "secret123");

    System.out.println("\n--- Test 2 : Identifiants incorrects ---");
    authService.login("admin", "badpassword");

    System.out.println("\n--- Test 3 : Paramètres invalides ---");
    authService.login("", "");

    System.out.println("\n--- Test 4 : Erreur système interceptée ---");
    authService.login("error_trigger", "123");
}
}

```

Module 10 : Spring Framework & Spring Core

Ce module est découpé en 3 chapitres essentiels :

1. **Chapitre 1 : Pourquoi Spring & Historique**
2. **Chapitre 2 : L'Inversion de Contrôle (IoC) & Injection de Dépendances (DI)**
3. **Chapitre 3 : Conteneur Spring, Beans & Architecture**

Chapitre 1 : Pourquoi Spring & Historique

1.1 Historique & Contexte

Au début des années 2000, le standard pour développer des applications d'entreprise Java était **EJB 2.x (Enterprise JavaBeans)** de Java EE. Bien que puissant, il présentait de lourds défauts :

- Modèle de composants excessivement complexe et lourd.
- Fort couplage du code métier aux classes/interfaces spécifiques de Java EE.
- Tests unitaires hors d'un serveur d'application presque impossibles.

En 2002, **Rod Johnson** publie l'ouvrage *Expert One-on-One J2EE Design and Development*, proposant une alternative plus légère basée sur des **POJO (Plain Old Java Objects)**. Ce code source est devenu la fondation du **Spring Framework** lancé en open source en 2003.

1.2 Pourquoi utiliser Spring ?

Spring s'est imposé comme le standard de fait grâce à plusieurs piliers :

- **Non-intrusif** : Le code métier n'est pas pollué par les classes du framework (on manipule de simples POJO).
- **Testabilité** : En isolant les composants, les tests unitaires avec JUnit et Mockito deviennent simples et rapides.
- **Gestion d'infrastructure** : Spring prend en charge la gestion du cycle de vie des objets, les transactions BDD, la sécurité et la configuration.
- **Écosystème modulaire** : Spring s'adapte aux besoins (Spring MVC, Spring Data, Spring Security, Spring Cloud).

Chapitre 2 : Inversion de Contrôle (IoC) & Injection de Dépendances

2.1 L'Inversion de Contrôle (*Inversion of Control - IoC*)

En programmation classique, une classe gère elle-même la création de ses dépendances via le mot-clé `new` :

Java

```
// ❌ Approche classique : Couplage fort
public class OrderService {
    private OrderRepository repository = new PostgresOrderRepository(); // Instanciation manuelle
}
```

Avec l'**Inversion de Contrôle**, le composant ne crée plus ses dépendances. C'est un tiers externe (le **Conteneur IoC**) qui se charge de créer, configurer et assembler les objets.

Le principe d'Hollywood : *"Ne nous appelez pas, nous vous appellerons."*

2.2 L'Injection de Dépendances (*Dependency Injection - DI*)

L'Injection de Dépendances est le mécanisme concret qui réalise l'IoC. Elle consiste à "injecter" les dépendances requises dans un composant, soit par **constructeur**, par **setter**, ou par **champ**.

Injection par Constructeur (Recommandée par Spring)

Java

```
public interface NotificationService {
    void notifyUser(String message);
}
```

@Service

```
public class OrderService {
```

```

private final NotificationService notificationService;

// ✅ Injection par constructeur : Dépendance explicite et immuable (final)
@Autowired // Optionnel si un seul constructeur est présent (depuis Spring 4.3)
public OrderService(NotificationService notificationService) {
    this.notificationService = notificationService;
}
}

```

Chapitre 3 : Conteneur Spring, Beans & Architecture

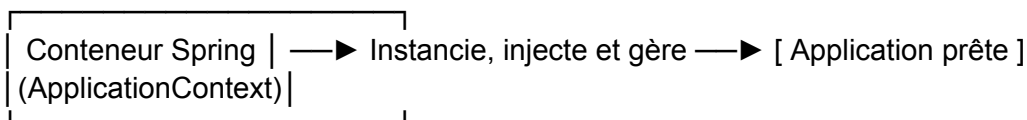
3.1 Le Conteneur Spring (*IoC Container*)

Le **Conteneur Spring** est le moteur au cœur du framework. Il est représenté par l'interface `ApplicationContext` (qui hérite de `BeanFactory`).

Ses missions :

1. Lire la configuration (annotations, Java Configuration ou XML).
2. Instancier les objets.
3. Injecter les dépendances entre eux.
4. Gérer leur cycle de vie.

[Configurations / Annotations]



3.2 Qu'est-ce qu'un Bean Spring ?

Dans Spring, un **Bean** est simplement un objet instancié, assemblé et géré par le Conteneur IoC de Spring.

Les annotations d'injection et de composants :

Spring utilise des annotations stéréotypes pour scanner et enregistrer automatiquement les Beans (`Component Scanning`) :

- **@Component** : Annotation générique pour marquer un objet comme Bean Spring.
- **@Service** : Spécialisation de `@Component` pour la couche métier (*Business Layer*).
- **@Repository** : Spécialisation pour la couche d'accès aux données (*DAO / Persistence*), traduit automatiquement les exceptions SQL en exceptions Spring.

- **@Controller / @RestController** : Spécialisation pour la couche d'exposition Web (API REST / MVC).

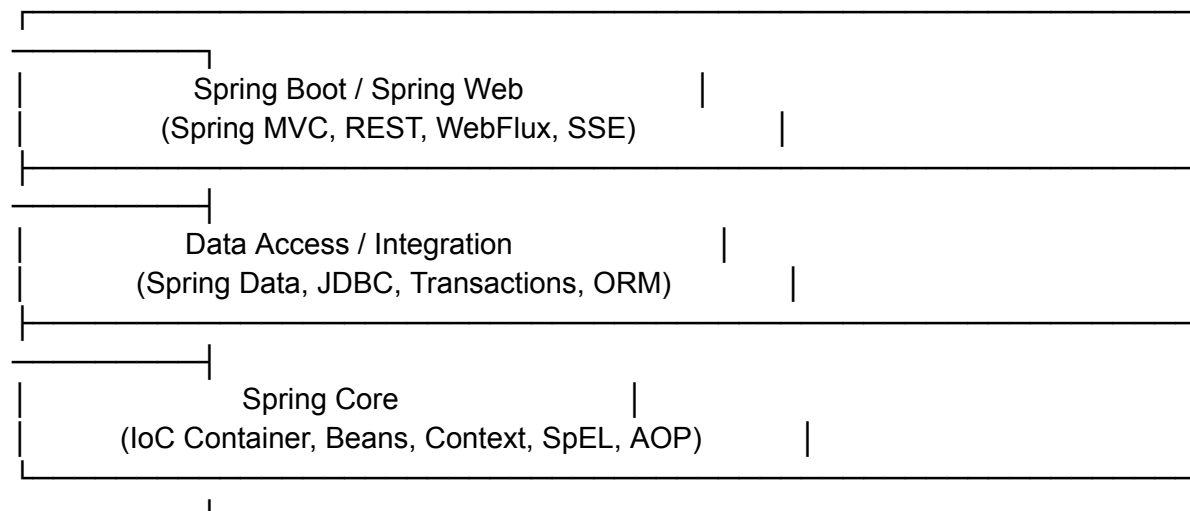
Portées des Beans (*Bean Scopes*) :

- **singleton (Par défaut)** : Une seule instance du Bean est créée par conteneur Spring.
- **prototype** : Une nouvelle instance est créée à chaque fois que le Bean est demandé.
- **request / session (Web)** : Une instance par requête HTTP ou par session utilisateur.

3.3 Architecture globale de Spring

Spring est structuré de manière modulaire en plusieurs couches :

Plaintext



TP Module 10 : Configuration d'un Conteneur IoC avec Spring Core

Objectif :

Créer une application console Java pure utilisant **spring-context** pour configurer le conteneur IoC et injecter automatiquement les dépendances entre les composants via des annotations.

Consignes du TP :

1. Ajouter la dépendance Maven **spring-context** dans le **pom.xml**.

2. Créer une interface `TaxCalculator` avec une méthode `double calculateTax(double amount)`.
3. Implémenter `FrenchTaxCalculator` annotée avec `@Component` (taux à 20%).
4. Créer la classe `InvoiceService` annotée avec `@Service` qui reçoit `TaxCalculator` par constructeur et fournit la méthode `double calculateTotal(double amount)`.
5. Créer une classe de configuration `AppConfig` annotée avec `@Configuration` et `@ComponentScan("...")`.
6. Dans `MainTP10`, démarrer un `AnnotationConfigApplicationContext`, récupérer le Bean `InvoiceService` et afficher le montant total TTC.

Correction détaillée du TP Module 10

1. Dépendance Maven (`pom.xml`)

XML

```
<dependencies>
  <!-- Dépendance Spring Core Context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.5</version>
  </dependency>
</dependencies>
```

2. L'Interface et l'Implémentation du Calculateur (`TaxCalculator.java` & `FrenchTaxCalculator.java`)

Java

```
public interface TaxCalculator {
    double calculateTax(double amount);
}
```

Java

```
import org.springframework.stereotype.Component;
```

`@Component`

```
public class FrenchTaxCalculator implements TaxCalculator {
```

```
    private static final double VAT_RATE = 0.20; // 20%
```

`@Override`

```
    public double calculateTax(double amount) {
        return amount * VAT_RATE;
    }
}
```

3. Le Service Métier (**InvoiceService.java**)

Java

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class InvoiceService {
```

```
    private final TaxCalculator taxCalculator;
```

```
    // Injection automatique du Bean TaxCalculator par le conteneur IoC
```

```
    public InvoiceService(TaxCalculator taxCalculator) {
```

```
        this.taxCalculator = taxCalculator;
```

```
    }
```

```
    public double calculateTotal(double amountHT) {
```

```
        double tax = taxCalculator.calculateTax(amountHT);
```

```
        return amountHT + tax;
```

```
    }
```

```
}
```

4. Classe de Configuration Spring (**AppConfig.java**)

Java

```
import org.springframework.context.annotation.ComponentScan;
```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
@ComponentScan(basePackages = "com.demo") // Scan automatique des annotations
```

```
@Component, @Service, etc.
```

```
public class AppConfig {
```

```
}
```

5. Classe Principale (**MainTP10.java**)

Java

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class MainTP10 {
```

```
    public static void main(String[] args) {
```

```
        // 1. Démarrage du Conteneur Spring (IoC Container)
```

```
        AnnotationConfigApplicationContext context =
```

```
            new AnnotationConfigApplicationContext(AppConfig.class);
```

```
        // 2. Récupération du Bean géré par Spring
```

```
        InvoiceService invoiceService = context.getBean(InvoiceService.class);
```

```
        // 3. Exécution de la méthode
```

```

double amountHT = 1000.0;
double totalTTC = invoiceService.calculateTotal(amountHT);

System.out.println("Montant HT : " + amountHT + " $");
System.out.println("Montant TTC (avec injection Spring) : " + totalTTC + " $");

// 4. Fermeture propre du conteneur
context.close();
}
}

```

Module 12 : Accès aux données avec Spring Data JPA & Hibernate

Ce module est découpé en 3 chapitres essentiels :

1. **Chapitre 1 : Les concepts de JPA, ORM et Hibernate**
2. **Chapitre 2 : Mappage des Entités & Relations (JPA Annotations)**
3. **Chapitre 3 : Spring Data JPA & Repositories (**JpaRepository**)**

Chapitre 1 : Les concepts de JPA, ORM et Hibernate

1.1 Qu'est-ce qu'un ORM et JPA ?

Dans une application orientée objet, on manipule des **objets Java** (ex: **Product**, **Order**).
 Dans une base de données relationnelle, on manipule des **tables, colonnes et clés étrangères**.

- **ORM (*Object-Relational Mapping*)** : C'est une technique qui permet de faire le pont automatiquement entre les objets Java et les tables SQL.
- **JPA (*Jakarta Persistence API*)** : C'est la **spécification standard** Java qui définit les interfaces et annotations pour l'ORM (comme **jakarta.persistence.Entity**).
- **Hibernate** : C'est l'**implémentation de référence** la plus populaire de JPA. C'est le moteur sous-jacent utilisé par défaut dans Spring Boot.

Plaintext

Spring Data JPA	<-- Abstraction haut niveau (Repositories)
JPA (Jakarta Persistence)	<-- Interface / Spécification standard
Hibernate	<-- Moteur ORM (Implémentation)

Driver JDBC & BDD	<-- PostgreSQL, MySQL, H2, etc.
-------------------	---------------------------------

Dépendances Maven (**pom.xml**) :

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<!-- Driver BDD (ex: PostgreSQL ou H2 en mémoire pour les tests) -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

Chapitre 2 : Mappage des Entités & Relations

Une **Entité** est une classe Java ordinaire (POJO) associée à une table dans la base de données via des annotations JPA (**jakarta.persistence.***).

2.1 Annotations de base sur une Entité

Java

```
import jakarta.persistence.*;
import java.time.LocalDateTime;
```

```
@Entity                // 1. Déclare la classe comme Entité JPA
@Table(name = "tb_products") // 2. Personnalise le nom de la table
public class Product {

    @Id                  // 3. Clé primaire
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-incrément SQL
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;

    @Column(nullable = false)
    private Double price;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
```

```
// Constructeurs, Getters et Setters obligatoires pour JPA
public Product() {} // Constructeur sans argument (Exigé par JPA)

public Product(String name, Double price) {
    this.name = name;
    this.price = price;
    this.createdAt = LocalDateTime.now();
}

// Getters et Setters...
}
```

2.2 Gestion des Relations entre Entités

JPA permet de traduire facilement les relations SQL de clés étrangères :

- **@ManyToOne** : Plusieurs éléments pointent vers un seul (ex: Plusieurs produits appartiennent à une seule **Category**). **C'est la relation la plus courante.**
- **@OneToMany** : Une entité est liée à plusieurs autres (ex: Une **Category** contient une liste de **Product**).
- **@OneToOne** : Relation 1 pour 1 (ex: Un **User** possède une **UserProfile**).
- **@ManyToMany** : Relation plusieurs à plusieurs (ex: Un **Product** a plusieurs **Tag**, et inversement).

Exemple pratique : Relation @ManyToOne / @OneToMany

```
Java
// Entité Côté "Many"
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY) // Chargement différé recommandé
    @JoinColumn(name = "category_id") // Nom de la colonne clé étrangère (FK)
    private Category category;

    // ...
}
```

```
Java
// Entité Côté "One"
@Entity
public class Category {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String name;

@OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
private List<Product> products = new ArrayList<>();

// ...
}

```

Chapitre 3 : Spring Data JPA & Repositories

3.1 La magie de **JpaRepository**

En Spring Data JPA, vous n'avez **plus besoin d'écrire de requêtes SQL/JPQL** pour les opérations CRUD de base. Il suffit de créer une interface qui hérite de **JpaRepository<T, ID>**.

```

Java
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Fournit gratuitement : save(), findById(), findAll(), deleteById(), count(), etc.
}

```

3.2 Derived Query Methods (Mots-clés de recherche automatique)

Spring Data JPA génère automatiquement la requête SQL sous-jacente simplement en lisant le nom de vos méthodes dans l'interface !

```

Java
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface ProductRepository extends JpaRepository<Product, Long> {

    // SQL : SELECT * FROM tb_products WHERE product_name = ?
    List<Product> findByName(String name);

    // SQL : SELECT * FROM tb_products WHERE price > ?
}

```

```

List<Product> findByPriceGreaterThan(Double minPrice);

// SQL : SELECT * FROM tb_products WHERE product_name LIKE %?%
List<Product> findByNameContainingIgnoreCase(String keyword);

// SQL : SELECT * FROM tb_products WHERE category_id = ? ORDER BY price DESC
List<Product> findByCategoryIdOrderByPriceDesc(Long categoryId);
}

```

3.3 Requêtes Personnalisées avec @Query (JPQL & Native)

Lorsque les requêtes deviennent complexes, on utilise l'annotation @Query :

```

Java
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface ProductRepository extends JpaRepository<Product, Long> {

    // Requête JPQL (sur les entités et objets Java)
    @Query("SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max")
    List<Product> findProductsInPriceRange(@Param("min") Double min, @Param("max")
    Double max);

    // Requête SQL native
    @Query(value = "SELECT * FROM tb_products WHERE product_name ILIKE
    %:keyword%", nativeQuery = true)
    List<Product> searchByNameNative(@Param("keyword") String keyword);
}

```

TP Module 12 : Système de Gestion de Catalogue & Catégories

Objectif :

Implémenter la couche de données (Entities + Repositories) d'un système de gestion de catalogue produits organisés par catégories, avec requêtes dérivées et personnalisées.

Consignes du TP :

1. Créer l'entité **Category** (id, name).
2. Créer l'entité **Product** (id, name, price, category avec @ManyToOne).
3. Créer l'interface **CategoryRepository**.

4. Créer l'interface **ProductRepository** comprenant :
 - Une méthode dérivée pour chercher les produits par nom exact.
 - Une méthode dérivée pour trouver les produits dont le prix est inférieur à une valeur max.
 - Une méthode avec **@Query** JPQL pour récupérer les produits d'une catégorie donnée filtrés par un prix minimum.
5. Écrire un **CommandLineRunner** dans l'application Spring Boot pour tester la création, l'association et la recherche de produits.

Correction détaillée du TP Module 12

1. L'Entité **Category.java**

Java

```
package com.demo.app.entity;
```

```
import jakarta.persistence.*;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
@Entity
```

```
@Table(name = "categories")
```

```
public class Category {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false, unique = true)
```

```
    private String name;
```

```
    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true)
```

```
    private List<Product> products = new ArrayList<>();
```

```
    public Category() {}
```

```
    public Category(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public Long getId() { return id; }
```

```
    public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }
```

```
    public List<Product> getProducts() { return products; }
```

```
}
```

2. L'Entité **Product.java**

Java

```
package com.demo.app.entity;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "products")
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false)
```

```
    private String name;
```

```
    @Column(nullable = false)
```

```
    private Double price;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "category_id")
```

```
    private Category category;
```

```
    public Product() {}
```

```
    public Product(String name, Double price, Category category) {
```

```
        this.name = name;
```

```
        this.price = price;
```

```
        this.category = category;
```

```
    }
```

```
    public Long getId() { return id; }
```

```
    public String getName() { return name; }
```

```
    public Double getPrice() { return price; }
```

```
    public Category getCategory() { return category; }
```

```
}
```

3. Les Repositories (**CategoryRepository.java** & **ProductRepository.java**)

Java

```
package com.demo.app.repository;
```

```
import com.demo.app.entity.Category;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface CategoryRepository extends JpaRepository<Category, Long> {
```

```
}
```

Java

```
package com.demo.app.repository;
```

```
import com.demo.app.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
```

```
import java.util.List;
```

```
public interface ProductRepository extends JpaRepository<Product, Long> {
```

```
    // 1. Derived Query Method
```

```
    List<Product> findByName(String name);
```

```
    // 2. Derived Query Method avec comparaison
```

```
    List<Product> findByPriceLessThan(Double maxPrice);
```

```
    // 3. Custom JPQL Query
```

```
    @Query("SELECT p FROM Product p WHERE p.category.id = :categoryId AND p.price  
    >= :minPrice")
```

```
    List<Product> findByCategoryAndMinPrice(@Param("categoryId") Long categoryId,  
    @Param("minPrice") Double minPrice);  
}
```

4. Le Runner de Test (**DataInitializer.java**)

Java

```
package com.demo.app;
```

```
import com.demo.app.entity.Category;
import com.demo.app.entity.Product;
import com.demo.app.repository.CategoryRepository;
import com.demo.app.repository.ProductRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
```

```
@Component
```

```
public class DataInitializer implements CommandLineRunner {
```

```
    private final CategoryRepository categoryRepository;
```

```
    private final ProductRepository productRepository;
```

```
    public DataInitializer(CategoryRepository categoryRepository, ProductRepository  
    productRepository) {
```

```
this.categoryRepository = categoryRepository;  
this.productRepository = productRepository;  
}
```

```
@Override
```

```
public void run(String... args) {  
    // 1. Création et sauvegarde d'une catégorie  
    Category tech = categoryRepository.save(new Category("Électronique"));  
  
    // 2. Création et sauvegarde de produits  
    productRepository.save(new Product("Ecran 4K", 350.0, tech));  
    productRepository.save(new Product("Clavier RGB", 80.0, tech));  
    productRepository.save(new Product("Souris Gamer", 45.0, tech));  
  
    // 3. Tests des requêtes  
    System.out.println("--- Produits de moins de 100 $ ---");  
    productRepository.findByPriceLessThan(100.0)  
        .forEach(p -> System.out.println(p.getName() + " - " + p.getPrice() + " $"));  
  
    System.out.println("\n--- Produits Électronique >= 50 $ (JPQL) ---");  
    productRepository.findByCategoryAndMinPrice(tech.getId(), 50.0)  
        .forEach(p -> System.out.println(p.getName() + " - " + p.getPrice() + " $"));  
}  
}
```